

CAF — Onboarding & External Context

Aggregated export — 2026-06-16. Source markdown in repo is unchanged.

Bundle id: `02-caf-onboarding-and-context`

Included: docs/EXTERNAL_CONTEXT_PACK.md

CAF Core — External context pack

Purpose: Tell another team, another repository, or an external LLM (ChatGPT, Claude, etc.) **everything it needs to understand, comment on, or re-implement CAF Core** — without pasting the full source tree.

How to use this file

1. Read this page first — it is the **table of contents and rules of engagement**.
2. Attach the **Tier 1 bundle** (below) to your LLM project or onboarding packet.
3. Add **Tier 2** docs when the topic is mimic, inputs, video, or publishing.
4. Attach **specific source files** only when asking for implementation-level changes.

Convention: Paths are from the **repository root**. Database schema is `caf_core`. The primary execution key is `task_id` scoped by `project_id`.

What CAF Core is (30-second version)

CAF (Content Automation Framework) is a **content automation pipeline**:

Signal Pack → **Candidates** → **Decision Engine** → **Content Jobs** → **LLM Drafts** → **QC** → **Rendering** → **Review** → **Publishing** → **Learning**

- **Source of truth:** PostgreSQL schema `caf_core`, especially `content_jobs` and `generation_payload`.
 - **Core API:** Fastify + TypeScript at repo root (`src/server.ts`).
 - **Review app:** Next.js client in `apps/review` — not the database of record.
 - **Media workers:** `services/renderer` (carousel PNGs), `services/video-assembly` (ffmpeg), `services/media-gateway` (combined port).
-

Tier 1 — Always include (general understanding)

Upload or paste these files together for **architecture, lifecycle, contracts, and invariants**:

#	File	What it gives the reader
1	AGENTS.md	Invariants, “do not break,” where to change behavior
2	docs/CAF_CORE_COMPLETE_GUIDE.md	Single-file merged reference (~500 lines)
3	docs/DOMAIN_MODEL.md	Entities, ID patterns, lifecycles (external copy of domain rules)
4	docs/ARCHITECTURE.md	Layer map, critical files, integration contracts
5	docs/LIFECYCLE.md	Run & job state machines
6	docs/TECH_STACK.md	Languages, services, third parties
7	docs/layers/README.md	Index to per-layer deep dives

Optional but recommended for rebuilds:

#	File	What it gives the reader
8	docs/REBUILD_FROM_DOCS.md	Step-by-step local/prod bootstrap
9	docs/DATABASE_SCHEMA.md	Table catalog grouped by domain
10	README.md	Quick start, CLI, deploy map
11	.env.example	Configuration surface (no secrets)
12	ENV_AND_SECRETS_INVENTORY.md	Full env var checklist

Tier 2 — Topic add-ons (attach when relevant)

Topic	Files
HTTP API examples	docs/API_REFERENCE.md, requests/caf-core.http
QC runtime	docs/QUALITY_CHECKS.md
Risk policies	docs/RISK_RULES.md
LLM prompt guidance	docs/GENERATION_GUIDANCE.md
Per-layer detail	docs/layers/*.md (10 files)
Video / HeyGen	docs/VIDEO_FLOWS.md, docs/HEYGEN_API_V3.md
Top-performer mimic	docs/MIMIC_FLOWS_COMPLETE_GUIDE.md, docs/MIMIC_IMAGE_FLOWS.md
Mimic text placement (future)	docs/MIMIC_TEXT_PLACEMENT_AUTOMATION.md
Creative Intelligence ingest	docs/CREATIVE_INTELLIGENCE.md

Inputs → signal pack	docs/CAF_INPUTS_PIPELINE_ROADMAP.md
Production deploy	docs/FLY_PRODUCTION_CHECKLIST.md
Secrets safety	docs/USER_INPUT_AND_SECRETS.md
Product pitch	docs/CAF_PRODUCT_PITCH.md
Complete product guide	docs/CAF_COMPLETE_PRODUCT_GUIDE.md
Shareable PDFs by topic	docs/export/pdf/*.pdf — SEE docs/export/README.md
Stakeholder overview	docs/PROJECT_OVERVIEW.md

Tier 3 — Source code (implementation questions only)

Do **not** upload the entire `src/` tree. Attach **3–15 files** for the specific task.

Goal	Start here
Run planning	src/decision_engine/, src/services/run-orchestrator.ts
Job execution	src/services/job-pipeline.ts
LLM generation	src/services/llm-generator.ts, src/repositories/flow-engine.ts
QC	src/services/qc-runtime.ts
Mimic	src/services/mimic-draft-prep.ts, mimic-carousel-render.ts, src/domain/mimic-payload.ts
Review contract	src/routes/v1.ts, src/routes/review-contract.test.ts
Publications	src/routes/publications.ts
Learning	src/services/learning-rule-selection.ts, src/routes/learning.ts
Config / flags	src/config.ts
Schema truth	migrations/*.sql (relevant migrations only)

Scale reference (~2026): ~113k lines TypeScript in `src/`, ~23k in `apps/review`, ~70 SQL migrations, ~140 test files.

System prompt template (copy into ChatGPT / Custom GPT)

You are helping with CAF Core – a Fastify + PostgreSQL content automation platform.

Pipeline funnel:

Signal Pack → Candidates → Decision Engine → Content Jobs → Drafts → QC
→ Render → Review → Publish → Learn

Rules:

- `task_id` + `project_id` are the primary keys for jobs, drafts, assets, reviews.
- `content_jobs.generation_payload` is the main JSON contract – treat changes as API design.
- The Review app (`apps/review`) is a client; Postgres `caf_core` is source of truth.
- Do not propose renaming `task_id`, `run_id` patterns, or lifecycle enums without explicit approval.
- QC writes `qc_result` only via `mergeGenerationPayloadQc` (`generation-payload-qc.ts`).
- HeyGen retries must use `hasActiveProviderSession` (`content-job-render-state.ts`).
- Learning lookups go through `learning-rule-selection.ts` facade.
- `risk_policies` + brand `banned_words` are enforced by QC; project `risk_rules` are NOT (see `risk-qc-status`).

When suggesting code: name exact file paths. When unsure which layer is affected, ask.

Prefer small, reviewable changes over repo-wide refactors.

Then attach **Tier 1** files from the table above.

What external readers can and cannot do from docs alone

Feasible from Tier 1 + 2	Requires source + tests
Explain architecture and workflow	Line-accurate refactors
Design a feature in the right layer	Prove dead-code removal is safe
Review API / contract changes	Re-implement every edge case
Bootstrap a new environment	Match 100% of admin UI behavior
Compare to another system	Replace LLM prompts verbatim

Honesty: docs/CAF_CORE_COMPLETE_GUIDE.md is a merged summary.
migrations/ and src/ win on conflicts.

Repository map (one screen)

```
CAF-Core/
├── src/                                # Core API (Fastify)
│   ├── server.ts                      # Entry
│   ├── config.ts                     # Env (Zod)
│   ├── routes/                       # HTTP (v1, runs, pipeline, admin, ...)
│   └── services/                     # Business logic (job-pipeline, llm-
generator, ...)
│   ├── decision_engine/              # Planning / scoring
│   ├── domain/                      # Typed payload readers, mimic contracts
│   └── repositories/                 # Postgres accessors
├── apps/review/                      # Next.js operator UI
├── services/
│   ├── renderer/                    # Carousel PNG worker
│   ├── video-assembly/              # ffmpeg worker
│   └── media-gateway/                # Combined gateway
├── migrations/                      # caf_core schema (versioned SQL)
├── docs/                             # All documentation
├── AGENTS.md                         # AI assistant invariants
├── .env.example                     # Config template
└── ENV_AND_SECRETS_INVENTORY.md
```

Related docs

Doc	Audience
REBUILD FROM DOCS.md	Engineers bootstrapping from scratch
DATABASE SCHEMA.md	DB tables and relationships
DOMAIN MODEL.md	IDs and lifecycles
CAF CORE COMPLETE GUIDE.md	Single-file full reference

Maintained as the canonical index for external context. Update this file when adding major docs or changing deploy topology.

Included: docs/REBUILD_FROM_DOCS.md

CAF Core — Rebuild from documentation

Purpose: Step-by-step guide for another team or repository to **stand up a working CAF Core stack** using only this repo's docs and code — without tribal knowledge.

Prerequisites: Node.js ≥ 20 , Docker (optional, for local Postgres), accounts for OpenAI and any media providers you enable.

Read first: [EXTERNAL CONTEXT PACK.md](#), [DOMAIN MODEL.md](#), [DATABASE SCHEMA.md](#).

What you are building

Component	Path	Port (default)	Required?
CAF Core API	repo root	3847	Yes
PostgreSQL	Docker or hosted	5432	Yes
Review app	apps/review	3000	Recommended
Carousel renderer	services/renderer	3333	For carousel flows
Video assembly	services/video-assembly	3334	For video stitch/mux
Media gateway	services/media-gateway	3300 / 8080	Optional (combines renderer + assembly)

Core alone can run planning, generation, QC, and API-backed review **without** media workers — but carousel/video render steps will fail until workers are reachable.

Phase 1 — Database

Option A: Docker Compose (local)

```
docker compose up -d
```

Use the `DATABASE_URL` from `.env.example` comments (typically `postgresql://postgres:postgres@localhost:5432/caf_core`).

Option B: Hosted Postgres (Supabase, Fly Postgres, etc.)

Create an empty database. Set `DATABASE_URL` to the connection string (pooler URI mode works).

Apply schema

```
cp .env.example .env
# Edit DATABASE_URL
npm install
npm run migrate
```

Migrations create schema `caf_core` and all tables. Re-running migrate is safe.

Optional seed data

```
npm run seed:demo          # Demo project
npm run seed:canonical-flows
npm run seed:flow-engine   # Flow definitions + prompts
```

Phase 2 — Core API minimum config

In `.env`, set at minimum:

Variable	Required	Notes
<code>DATABASE_URL</code>	Yes	Postgres connection
<code>OPENAI_API_KEY</code>	Yes for LLM	Generation, QC helpers, mimic copy
<code>OPENAI_MODEL</code>	Recommended	Default chat model

For a **smoke test** without rendering:

```
npm run dev
curl http://localhost:3847/health
```

Recommended for full pipeline

Variable	Purpose
RENDERER_BASE_URL	Carousel worker (default http://localhost:3333)
VIDEO_ASSEMBLY_BASE_URL	ffmpeg worker (default http://localhost:3334)
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ASSETS_BUCKET	Asset uploads
HEYGEN_API_KEY	Avatar / video agent flows
MIMIC_IMAGE_ENABLED=1 + provider keys	Top-performer mimic render

Full list: `.env.example` , `ENV_AND_SECRETS_INVENTORY.md` , `src/config.ts`
(Zod schema = source of truth).

Phase 3 — Media workers

Carousel renderer

```
cd services/renderer
npm install
# Optional: CAF_TEMPLATE_API_URL=http://localhost:3847 (pull HBS from Core)
npm start
# Listens on PORT (default 3333)
```

Core serves templates at `GET /api/templates/*` from `CAROUSEL_TEMPLATES_DIR` (see `src/routes/renderer-templates.ts`).

Video assembly

```
cd services/video-assembly
npm install
# Set SUPABASE_* if uploading outputs
npm start
# Listens on PORT (default 3334)
```

Media gateway (production pattern)


```
cd services/media-gateway
npm install
node server.js
```

Child processes inherit env — set Supabase and mux vars on the gateway host. See `services/media-gateway/` and `fly.toml` variants.

Phase 4 — Review app

```
cd apps/review
npm install
```

Create `apps/review/.env.local` :

```
CAF_CORE_URL=http://localhost:3847
# If Core uses auth:
# CAF_CORE_TOKEN=<same as CAF_CORE_API_TOKEN>
# Optional:
# RENDERER_BASE_URL=http://localhost:3333
# PROJECT_SLUG=SNS
```

```
npm run dev
# http://localhost:3000
```

Review is a **client** — all job state comes from Core APIs.

Phase 5 — End-to-end workflow

Path A: Legacy XLSX intake

```
npm run start-run:xlsx -- path/to/signals.xlsx --project SNS
```

Or `POST /v1/signal-packs/upload` (multipart).

Path B: Inputs pipeline (evidence → signal pack)

1. Admin UI: `/admin/inputs` — upload evidence XLSX or run scrapers
2. `/admin/processing` — insights, profile, build signal pack
3. Create run linked to signal pack

See `docs/CAF_INPUTS_PIPELINE_ROADMAP.md`.

Run lifecycle (API)

1. Create run with `signal_pack_id` → status `CREATED`
2. `POST /v1/runs/:project_slug/:run_id/jobs` — materialize `planned_jobs_json`
3. `POST /v1/runs/:project_slug/:run_id/start` — plan + insert `content_jobs`
4. `POST /v1/runs/:project_slug/:run_id/process` — generate + QC (background)
5. `POST /v1/runs/:project_slug/:run_id/render` — render `GENERATED jobs`
6. Review via Review app or `/v1/reviews`
7. `POST /v1/publications/:project_slug/...` — publish placements

CLI alternative: `npm run process-run -- <run_id> --project SNS`

Phase 6 — Production deploy

Target	Config
Core	<code>fly.toml</code> , <code>Dockerfile</code> at repo root
Media gateway	<code>services/media-gateway/fly.toml</code>
Review	<code>apps/review/vercel.json</code> — Vercel

Checklist: `docs/FLY_PRODUCTION_CHECKLIST.md`

Production essentials:

- `CAF_CORE_REQUIRE_AUTH=1` + `CAF_CORE_API_TOKEN`
- `CAF_PUBLIC_URL` — public Core URL for asset links
- `RENDERER_BASE_URL` / `VIDEO_ASSEMBLY_BASE_URL` — stable worker URLs

- Run `npm run migrate` on deploy or enable `CAF_RUN_MIGRATIONS_ON_START`
-

Phase 7 — Verify the stack

Check	Command / URL
Core health	GET /health
Rendering deps	GET /health/rendering
Review → Core	Review app /api/health/core
Renderer	GET {RENDERER_BASE_URL}/health
DB migrations	SELECT * FROM caf_core.schema_migrations ORDER BY version
Tests	npm test (Vitest)

What docs cannot replace

To **fully re-implement** behavior identical to this repo, you still need:

- **Source code** in `src/services/job-pipeline.ts` , `llm-generator.ts` , mimic modules, etc.
- **Flow engine seed data** — prompt templates and output schemas in DB
- **Migration history** — incremental schema evolution in `migrations/`
- **Integration tests** — `src/**/*.test.ts` , `src/routes/review-contract.test.ts`

For **architectural parity**, Tier 1 docs in [EXTERNAL CONTEXT PACK.md](#) are sufficient. For **behavioral parity**, treat this repository as the reference implementation.

Troubleshooting

Symptom	Likely cause
start fails — no candidates	Run POST .../jobs first to materialize planned_jobs_json
Jobs stuck RENDERING	HeyGen poll / missing HEYGEN_API_KEY / RenderNotReadyError retry
Carousel render timeout	Renderer not running; check RENDERER_BASE_URL, CAROUSEL_RENDERER_SLIDE_TIMEOUT_MS

QC passes but risk expected	Project <code>risk_rules</code> not enforced — use <code>risk_policies</code> or brand bans
Review 401	<code>CAF_CORE_TOKEN</code> mismatch with Core <code>CAF_CORE_API_TOKEN</code>
Mimic disabled	<code>MIMIC_IMAGE_ENABLED</code> not set; flow type not in allowed list

See also

- [README.md](#) — quick start
- [API REFERENCE.md](#) — HTTP examples
- [TECH STACK.md](#) — stack detail
- [CAF CORE COMPLETE GUIDE.md](#) — merged reference

Included: docs/DOMAIN_MODEL.md

CAF Core — Domain model

Purpose: External-facing copy of the CAF domain rules (IDs, entities, lifecycles, joins). Cursor agents also use `.cursor/rules/caf-domain-model.mdc` ; **this file is for humans, ChatGPT, and other repositories.**

Pipeline funnel:

Signal Pack → Candidates → Decision Engine → Content Jobs → Drafts → Rendering → Review → Publishing → Learning

Entity hierarchy

Entity	Description
Project	Named content brand (e.g. SNS). Container for strategy, flows, prompts, learning rules. Stored in <code>caf_core.projects</code> ; URLs use <code>slug</code> .
Run	One execution cycle for a project, tied to a signal pack. Groups all work from a single evidence window.
Signal pack	Research bundle attached to a run. Curated planner rows live in <code>jobs_json</code> (dual-written with legacy <code>ideas_json</code>). Legacy intake also used <code>overall_candidates_json</code> .
Candidate	One idea × one flow type. Built in memory from signal pack rows × enabled flows. Scored by the decision engine. Not persisted as first-class DB rows in current app code (historical <code>caf_core.candidates</code> table exists but is not the planning source of truth).

Content job	Atomic executable unit. Created from planned candidates. Central entity — rendering, review, publishing, and learning key off <code>task_id</code> .
Job draft	One LLM generation attempt for a job (<code>attempt_no</code> , <code>revision_round</code>).
Asset	Produced media (carousel image, video, audio, subtitles) linked to a job.
Editorial review	Human approval / rejection / edit decision on a job.
Diagnostic audit	Machine-generated quality evaluation.
Performance metric	Post-publication outcome data.
Learning rule	Structured insight that changes future behavior (prompt selection, routing, scoring).
Experiment	Controlled intervention to measure impact of changes.
Publication placement	Scheduled or completed publish intent per platform.

Inputs pipeline entities (upstream of signal packs)

Entity	Description
Evidence import	XLSX or scraper ingest → <code>inputs_evidence_imports</code> + <code>inputs_evidence_rows</code> .
Evidence insights	Row-level LLM/vision passes → <code>inputs_evidence_row_insights</code> (broad, top-performer tiers).
Ideas	Structured idea list → <code>ideas</code> , <code>signal_pack_ideas</code> , <code>inputs_ideas</code> .
Creative Intelligence	Archived top-performer media + vision analysis → <code>creative_*</code> tables.

See `docs/CAF_INPUTS_PIPELINE_ROADMAP.md` and `docs/CREATIVE_INTELLIGENCE.md`.

ID conventions

Entity	Field	Pattern	Example
Run	<code>run_id</code>	<code>{PROJECT}_{period}</code>	<code>SNS_2026W0</code>
Candidate	<code>candidate_id</code>	<code>{run_id}_{platform}_{NNNN}</code> OR <code>{base}_{flow_type}</code>	<code>SNS_2026W0</code>
Job	<code>task_id</code>	<code>{run_id}_{platform}_{flow_type}_row{NNNN}_{variation}</code>	<code>SNS_2026W0</code>
Draft	<code>draft_id</code>	<code>d_{random12}</code>	<code>d_mn398eop</code>

Asset	asset_id	{candidate_id}__{ASSET_TYPE}_v{version}	SNS_2026W0
Scene	scene_id	{task_id}__scene_{NN}	...row0012

task_id is the main execution key. Downstream tables (`job_drafts` , `assets` , `editorial_reviews` , `job_state_transitions` , `diagnostic_audits` , `performance_metrics` , `auto_validation_results`) reference it.

Relationships (text joins, not UUID FKs)

- `content_jobs.run_id` = `runs.run_id` (same text key, scoped by `project_id`)
- `content_jobs.candidate_id` = decision-engine candidate id (text)
- `job_drafts.task_id` = `content_jobs.task_id`
- Cross-table joins use **(project_id, task_id)** or **(project_id, run_id)**

Run lifecycle

CREATED → PLANNING → PLANNED → GENERATING → RENDERING → REVIEWING → COMPLETED

CANCELLED

↘ FAILED /

Planning prerequisite: Materialize planner rows into `runs.planned_jobs_json` (canonical; dual-written with legacy `candidates_json`) via `POST /v1/runs/:project_slug/:run_id/jobs` before `POST .../start` .

Job lifecycle

Typical path:

PLANNED → GENERATING → GENERATED → (QC) → RENDERING → IN_REVIEW → APPROVED | REJECTED | NEEDS_EDIT

QC may route to `BLOCKED` , `QC_FAILED` , or short-circuit to `REJECTED` / `NEEDS_EDIT` . Video jobs may remain `RENDERING` during HeyGen/Sora polls.

Full detail: `docs/LIFECYCLE.md` .

Editorial decisions

`APPROVED` | `NEEDS_EDIT` | `REJECTED`

Critical JSON contract: `generation_payload`

On `caf_core.content_jobs` . Integration hub for pipeline, Review, and admin.

Slice	Canonical helpers
<code>generated_output</code>	<code>src/domain/generation-payload-output.ts</code>
<code>qc_result</code>	<code>src/domain/generation-payload-qc.ts</code> — Write via <code>mergeGenerationPayloadQc</code> only
<code>render_state</code>	<code>src/domain/content-job-render-state.ts</code> — <code>hasActiveProviderSession</code> for HeyGen idempotency
Mimic render truth	<code>mimic_v1</code> ON <code>generation_payload</code> (<code>src/domain/mimic-payload.ts</code>)
Carousel mimic review	<code>mimic_carousel_package</code> — only <code>FLOW_TOP_PERFORMER_MIMIC_CAROUSEL</code> ; not <code>carousel_package</code>

Invariants (do not break without explicit approval)

- `task_id` remains the primary execution key across all job-related tables.
- Do not rename or restructure the ID hierarchy casually.
- Preserve parent-child traceability (scene bundles: `parent_id` / `parent_candidate_id`).
- New tables follow the **text-ID join** pattern.
- Status values stay compatible with existing state machines.
- `content_jobs` is the source of truth for job state.

Flow type families (planning / pipeline)

Family	Examples	Notes
Carousel	<code>FLOW_CAROUSEL</code>	Standard HBS renderer path
Video	<code>FLOW_*</code> video kinds, HeyGen, scene assembly	See <code>flow-kind.ts</code>

Product video	FLOW_PRODUCT_*	Product-focused video flows
Mimic	FLOW_TOP_PERFORMER_MIMIC_IMAGE, FLOW_TOP_PERFORMER_MIMIC_CAROUSEL	Requires MIMIC_IMAGE_ENABLED
Image product	FLOW_IMG_*	Registered; not fully wired to generation
Offline	Various	Excluded from pipeline — offline-flow-types.ts

See also

- [EXTERNAL CONTEXT PACK.md](#) — what to upload to external LLMs
- [DATABASE SCHEMA.md](#) — table catalog
- [LIFECYCLE.md](#) — full state machines
- [ARCHITECTURE.md](#) — where code lives

Included: docs/DATABASE_SCHEMA.md

CAF Core — Database schema reference

Purpose: Table catalog for engineers, external re-implementers, and LLMs that need to understand **where state lives** without reading every migration.

Schema name: `caf_core`

Migrations: `migrations/*.sql` (applied via `npm run migrate` or `CAF_RUN_MIGRATIONS_ON_START`)

Tracking table: `caf_core.schema_migrations`

Join convention: Prefer `(project_id, task_id)` or `(project_id, run_id)` text keys — not UUID foreign keys across aggregates.

Core pipeline tables

Table	Role
<code>projects</code>	Tenants; <code>slug</code> used in URLs
<code>project_system_constraints</code>	System-wide caps per project
<code>runs</code>	Run header; <code>run_id</code> , <code>status</code> , <code>signal_pack_id</code> , <code>planned_jobs_json</code> / <code>candidates_json</code> , <code>context_snapshot_json</code> , <code>plan_summary_json</code>

signal_packs	Research bundles; jobs_json, ideas_json, overall_candidates_json, platform summaries, visual_guidelines_pack_v1
content_jobs	Central table — task_id, status, generation_payload, render_state, QC columns
job_drafts	LLM attempts per task_id
assets	Rendered media linked to jobs
job_state_transitions	Audit log of status changes
decision_traces	Planning decision snapshots
candidates	Historical — not current planning source of truth

Review, QC, and validation

Table	Role
editorial_reviews	Human decisions (APPROVED, NEEDS_EDIT, REJECTED)
diagnostic_audits	Machine diagnostics on jobs
auto_validation_results	Automated validation records
validation_events	Validation event stream
llm_approval_reviews	Post-approval LLM review scores + upstream_recommendations
run_output_reviews	Run-level output reviews

Flow engine & QC metadata

Table	Role
flow_definitions	Flow types, routing, QC checklist names
prompt_templates	Prompt bodies and versions
output_schemas	JSON schema for LLM output validation
carousel_templates	Carousel HBS template metadata
qc_checklists	QC check rows (required_keys, regex, ...)
risk_policies	Keyword risk policies (scoped by applies_to_flow_type)
prompt_labs_overrides	Prompt lab experiment overrides

Project configuration

Table	Role
strategy_defaults	Strategy JSON per project
brand_constraints	Brand voice, banned words
platform_constraints	Per-platform limits
risk_rules	Project risk rows (not enforced by QC today — see RISK_RULES.md)
allowed_flow_types	Enabled flows per project
heygen_config	HeyGen avatar / voice settings
project_product_profile	Product video profile
project_brand_assets	Brand images for HeyGen / mimic
project_carousel_templates	Project-specific carousel templates
project_integrations	Meta FB/IG IDs, tokens metadata
reference_posts, viral_formats	Reference content config

Learning & performance

Table	Role
learning_rules	Active/pending rules (rule_family, action_type, scopes)
suppression_rules	Planning suppression
learning_observations	Evidence observations (incl. upstream rec items)
learning_hypotheses, learning_hypothesis_trials	Experiment tracking
learning_insights	Compiled insights
learning_generation_attribution	Which rules affected a generation
performance_metrics	Post-publish metrics
performance_ingestion_batches	Batch ingest metadata
run_content_outcomes	Run-level outcome summaries

Publishing

Table	Role
publication_placements	Publish intent + status (draft → published / failed)

Inputs & evidence pipeline

Table	Role
inputs_evidence_imports	XLSX / ingest batch header
inputs_evidence_rows	Normalized evidence rows
inputs_evidence_row_insights	LLM/vision insights per row (tiers: broad, top_performer_*)
inputs_evidence_packs	Packaged evidence for processing
inputs_processing_profiles	Scoring criteria, models, caps
inputs_source_rows	Workbook source registry
inputs_scraper_config, inputs_scraper_runs	Apify scraper config + runs
inputs_idea_lists, inputs_ideas	Idea list intake
ideas, idea_grounding_insights	Structured ideas + grounding
signal_pack_ideas, signal_pack_selected_ideas	Ideas linked to signal packs
insights_packs	Insights pack metadata
qc_flow_profiles	Per-flow QC profile config
evidence_media_assets	Archived media for evidence rows

Creative Intelligence (top-performer mimic upstream)

Table	Role
creative_source_assets	Ingested top-performer media
creative_visual_analyses	Vision analysis per asset
creative_insights	Aggregated creative insights
creative_carousel_mimic_templates	Carousel mimic template library

Operations & audit

Table	Role
api_call_audit	External API call log + cost estimates
prompt_versions	Prompt version history
experiments	A/B prompt experiments
generation_counters	Generation counters per scope

Key columns on `content_jobs`

Column	Purpose
task_id	Primary execution key (with project_id)
run_id	Parent run (text)
candidate_id	Planner candidate id (text)
flow_type, platform	Routing / QC scope
status	Job lifecycle state
qc_status, recommended_route	QC outcome (also inside payload)
generation_payload	Main JSON contract — prompts, LLM output, QC, mimic, publish URLs
render_state	Provider session state (HeyGen video_id, etc.)
scene_bundle_state	Scene assembly progress
review_snapshot	Snapshot for review UI

Key columns on `runs`

Column	Purpose
run_id	Human-readable run key (with project_id)
signal_pack_id	Attached research bundle
status	Run lifecycle
planned_jobs_json	Materialized planner rows (canonical)
candidates_json	Legacy alias (dual-written)

ER sketch (logical, not SQL FKs)

```
erDiagram
    projects ||--o{ runs : has
    projects ||--o{ signal_packs : has
    runs ||--o{ signal_packs : attaches
    runs ||--o{ content_jobs : plans
    content_jobs ||--o{ job_drafts : generates
    content_jobs ||--o{ assets : produces
    content_jobs ||--o{ editorial_reviews : reviewed
    content_jobs ||--o{ publication_placements : publishes
    projects ||--o{ learning_rules : learns
```

Bootstrap order for a fresh database

1. `npm run migrate` — applies all `migrations/*.sql` in order
2. `npm run seed:demo` or `npm run seed:canonical-flows` — optional demo project + flows
3. `npm run seed:flow-engine` — flow definitions and prompts (as needed)
4. Configure project via admin or `import:project-csv`

See also

- [DOMAIN MODEL.md](#) — IDs and lifecycles
- [REBUILD FROM DOCS.md](#) — full bootstrap guide
- [ARCHITECTURE.md](#) — which repositories read/write these tables
- `migrations/001_caf_core_schema.sql` — original core DDL

Table list aligned with migrations through 069. When in doubt, `grep CREATE TABLE in migrations/`.

Included: AGENTS.md

AGENTS.md — Guidance for AI coding assistants

This file helps **Cursor agents** (and humans) work safely and efficiently in **CAF Core**.

Project identity

- **Repo:** CAF Core — Fastify + PostgreSQL backend for a **content automation pipeline** (signals → jobs → LLM → QC → render → review → publish → learning).
- **Source of truth:** Postgres schema **caf_core** + **content_jobs** rows, especially **generation_payload** .
- **Review app** (`apps/review`) is a **client** of Core APIs, not the authority for job state.

Always read first

Resource	Why
<code>docs/EXTERNAL_CONTEXT_PACK.md</code>	Tiered doc bundle for ChatGPT / other repos / rebuilds
<code>docs/DOMAIN_MODEL.md</code>	ID conventions , <code>task_id</code> joins, lifecycle states (external copy)
<code>.cursor/rules/caf-domain-model.mdc</code>	Same domain rules (Cursor always-on)
<code>docs/ARCHITECTURE.md</code>	Layers, critical files, QC/learning split
<code>src/config.ts</code>	Env flags (Zod); defaults for validation, human review after QC, URLs

Invariants — do not break without explicit user approval

1. **task_id** is the primary execution key across **job_drafts** , **assets** , **editorial_reviews** , **job_state_transitions** , etc., scoped by **project_id** .
2. Prefer joins on **(project_id, task_id)** or **(project_id, run_id)** using **text IDs** as documented in **caf-domain-model**.
3. **content_jobs.generation_payload** is the main JSON contract; coordinate changes with pipeline, Review, and admin consumers. Use **mergeGenerationPayloadQc** (`src/domain/generation-payload-qc.ts`) to write the **qc_result** slice — it is the only sanctioned writer for that subset.
4. **HeyGen retry**: avoid double-submitting renders when **render_state** already holds **video_id** / **session_id** . Use **hasActiveProviderSession** from **src/domain/content-job-render-state.ts** — do not re-implement the check inline.

5. **Learning lookups go through the facade** at `src/services/learning-rule-selection.ts` — `getLearningRulesForPlanning` for planning, `getLearningContextForGeneration` for prompts. Do not import `listActiveAppliedLearningRules` or `compileLearningContexts` directly from new code.
6. **QC risk enforcement today:** `risk_policies` (scoped per `flow_type` via `applies_to_flow_type`) + `brand_constraints.banned_words`. Project `risk_rules` are **not** enforced by `qc-runtime.ts`; **GET** `/v1/projects/:slug/risk-qc-status` is the canonical honesty endpoint.
7. **Review app** `/v1/contract` is covered by `src/routes/review-contract.test.ts`. If you rename or remove a listed path, update that test in the same change.
8. **Typed payload readers** — prefer the helpers in `src/domain/` over ad-hoc casts:
 - `pickGeneratedOutput` / `hasGeneratedOutput` for `generation_payload.generated_output`
 - `pickRenderState` / `hasActiveProviderSession` / `isMidProviderPhase` for `render_state`
 - `pickStoredQcResult` / `mergeGenerationPayloadQc` for `qc_result`
9. **Structured logs:** when adding new pipeline logging, prefer `logPipelineEvent(level, stage, message, { run_id, task_id, job_id, ... })` from `src/services/pipeline-logger.ts` over `console.*`. Correlation fields are how we trace a single job across stages.
10. **Upstream recommendations** (`src/domain/upstream-recommendations.ts`) are the structured shape the post-approval LLM reviewer emits. Writers must use `parseUpstreamRecommendations` (tolerant) and persist via `insertLlmApprovalReview`; each parsed item is also logged as its own `learning_observation` — do not bypass either when extending the reviewer.
11. **Run context snapshot** (`src/services/run-context-snapshot.ts`) is the canonical record of "what we actually generated with" (prompt versions + brand/strategy slice + learning fingerprints). Persist via `setRunContextSnapshot`. Snapshot failures are logged but must never abort a run.
12. **Mimic carousel package** — only `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL` uses `mimic_carousel_package`; do not conflate with `FLOW_CAROUSEL` / `carousel_package`. Render truth is `mimic_v1` on `generation_payload`. Mimic LLM prompts must filter signal pack to the job's single planned idea (`llm-creation-pack-budget.ts`, `mimicFlowOnly: true`).

Where to change behavior

Goal	Start here
Run planning, scoring, suppression	<code>src/decision_engine/</code> , <code>src/services/run-orchestrator.ts</code>
LLM prompts & creation pack	<code>src/services/llm-generator.ts</code> , <code>src/repositories/flow-engine.ts</code>
QC checks / risk keywords	<code>src/services/qc-runtime.ts</code> , checklist rows, scoped risk_policies via <code>listRiskPoliciesForJob</code>
Job lifecycle & render	<code>src/services/job-pipeline.ts</code>
Top-performer mimic (prep, render, modes)	<code>src/services/mimic-draft-prep.ts</code> , <code>mimic-carousel-render.ts</code> , <code>mimic-image-job.ts</code> , <code>mimic-mode-classifier.ts</code> , <code>src/domain/mimic-payload.ts</code>
Human review & rework	<code>src/routes/v1.ts</code> , <code>src/services/rework-orchestrator.ts</code>
Publications	<code>src/routes/publications.ts</code>
Learning APIs & compiled guidance	<code>src/routes/learning.ts</code> , facade at <code>src/services/learning-rule-selection.ts</code>
New HTTP routes	Register in <code>src/server.ts</code> (pattern: <code>register*Routes</code>)

Conventions

- **TypeScript ESM** — imports use `.js` extensions in source (Node ESM).
- **Tests:** Vitest — `npm test` ; colocate `*.test.ts` under `src/` .
- **Migrations:** SQL in `migrations/` ; applied via `src/cli/migrate.ts` / startup.
- **Avoid** drive-by refactors in files unrelated to the task; match existing style.

Commands (from repo root)

```
npm run dev          # Core API (watch)
npm run migrate      # Apply DB migrations
npm test             # Vitest
npm run process-run -- <run_id|uuid> [--project SLUG]
```

Review app: `cd apps/review && npm run dev` (needs `CAF_CORE_URL`).

Documentation map

Doc	Audience
docs/CAF_PRODUCT_PITCH.md	Leadership / investors / evaluators
docs/CAF_COMPLETE_PRODUCT_GUIDE.md	Complete product guide — what CAF is and does
docs/EXTERNAL_CONTEXT_PACK.md	ChatGPT / external repos — what to upload, system prompt
docs/REBUILD_FROM_DOCS.md	Engineers bootstrapping from scratch
docs/DATABASE_SCHEMA.md	Postgres table catalog
docs/DOMAIN_MODEL.md	Entities, IDs, lifecycles
README.md	Quick start, API index, deploy
docs/CAF_CORE_COMPLETE_GUIDE.md	Single-file full project reference
docs/PROJECT_OVERVIEW.md	Stakeholders / onboarding
docs/ARCHITECTURE.md	Engineers
docs/LIFECYCLE.md	Run & job states
docs/TECH_STACK.md	Stack & services
docs/layers/README.md	Per-layer deep dives
docs/QUALITY_CHECKS.md	QC runtime
docs/GENERATION_GUIDANCE.md	Prompt guidance
docs/RISK_RULES.md	Risk policies vs project risk_rules
docs/MIMIC_FLOWS_COMPLETE_GUIDE.md	Top-performer mimic (full); docs/MIMIC_IMAGE_FLOWS.md quick ref
docs/MIMIC_TEXT_PLACEMENT_AUTOMATION.md	Future mimic text placement QA (post-render composite loop) — read before automating
docs/CREATIVE_INTELLIGENCE.md	Top-performer ingest upstream of mimic
docs/API_REFERENCE.md	HTTP examples

Optional rules (Cursor)

- `scene-assembly-n8n-legacy.mdc` — when touching scene/n8n-legacy paths
- `mimic-signal-pack-llm-filter.mdc` — mimic LLM creation pack; single-idea `signal_pack` filter

- `mimic-carousel-package.mdc` — `mimic_carousel_package` vs `FLOW_CAROUSEL / carousel_package`
- `mimic-text-placement-automation.mdc` — **before automating mimic text placement**: post-render composite QA loop, HTML overlay invariants, `docai_layer_positions` schema
- `caf-domain-model.mdc` — **alwaysApply**

If the user's request conflicts with **domain invariants**, **ask** or **surface the tradeoff** before renaming IDs or changing status enums.